

Reviewer Pack — Minimal Rank of Real Algebraic Varieties over Boolean Functi...

Entry 1e3203759a39 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 08:10:09 UTC

Minimal Rank of Real Algebraic Varieties over Boolean Functions vs AC0 Parity Circuit Size

Entry ID: 1e3203759a39 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-25 08:10:09 UTC

1. Conjecture

Field A (mathematical branch): Real Algebraic Geometry **Field B** (complexity object): Complexity Theory: AC⁰ Parity Circuit Complexity

Statement:

{‘content’: ‘For every Boolean function f with n variables, the minimal rank of its associated real algebraic variety V_f is $\Theta(\log(n)/\log(\log(n)))$ whenever f computes PARITY.’}

Rationale (proposer’s reasoning):

{‘content’: ‘Real algebraic geometry provides a structured way to represent Boolean functions as varieties over the reals. If this invariant grows logarithmically with the number of variables for PARITY, it could expose an intrinsic structure that is not captured by classical circuit complexity measures.’}

Taxonomy category: AC0_PARITY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ef9d6e0fa7a902cd

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the mean dimension of the associated real algebraic varieties for PARITY functions is within ± 3 units of $\Theta(\log(n)/\log(\log(n)))$ across 30 random seeds, and all seeds produce a dimension ≤ 10 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "real algebraic geometry" AND "AC0 parity circuit complexity" - "minimal rank" AND "real algebraic variety" AND "Boolean function" - "PARITY problem" AND "real algebraic geometry" AND "circuit size"

Top relevant hits considered: - [s2:2502.10590] Optimal Bounds for the Number of Pieces of Near-Circuit Hypersurfaces - [s2:0ebb3361a3ada7e3cf32a5d479c45ffa7cd44880] Interactions between Representation Theory , Quantum Field Theory , Category Theory and Quantum Information Theory Abstr

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.0s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 5 # Start with small n and increase for more trials
    max_n = 40
    metric_values = []

    while len(metric_values) < 30 and n <= max_n:
        f = [random.choice([0, 1]) for _ in range(n)]
        if sum(f) % 2 == 0: # Ensure the function computes PARITY
            V_f = set()
            for i in range(2**n):
                x = [(i >> j) & 1 for j in range(n)]
                if f == [x[j] ^ (sum(x[:j]) % 2) for j in range(n)]:
                    V_f.add(tuple(x))

            dimension = len(V_f)
            metric_values.append(dimension)
        n += 5

    mean_value = sum(metric_values) / len(metric_values)
    conjecture_holds = abs(mean_value - math.log(n, 2) / math.log(math.log(n, 2), 2)) <= 3
    counterexample = "" if conjecture_holds else f"mean={mean_value}, expected=0(log({n})/log(log({n})))

    return {
        "metric_name": "dimension",
        "metric_value": mean_value,
        "instances_tested": len(metric_values),
    }
```

```

        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or list(range(2, 30)) # Default to first 29 primes

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means we cannot confirm whether the mean dimension of the associated real algebraic varieties for PARITY functions is within the specified bounds. | next: Run the test again with increased time limits to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	10010
2	preregistration	ollama_remote	glm4:latest	0	0	5593
3	novelty	ollama_remote	glm4:latest	0	0	4815
4	novelty	ollama_remote	glm4:latest	0	0	6014
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11390
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8240
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8300
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12168
9	judge	ollama_remote	glm4:latest	0	0	12940

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 79470 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/1e3203759a39.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/1e3203759a39.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/1e3203759a39.tar.gz* (if generated)